DocMind - Project Overview & Learning Notes

What We're Building

DocMind — An AI-powered Document Intelligence Platform that lets users upload PDFs, extract content (text, tables, images), and chat with their documents using natural language.

Why This Project

- Combines everything learned from the freeCodeCamp "Production RAG with LangChain & Vector Databases" course
- Modeled after Akshat Jiwrajka's Doc-Analyzer (ranked #1 from IIT project analysis) but with better engineering
- Covers the full production stack: API, agent, retrieval, security, observability

Architecture Decisions (and Why)

Tech Stack

Component	Our Choice	Akshat's Choice	Why Ours is Better
Vector DB	PGVector (Supabase)	MongoDB + manual cosine	PGVector does similarity search at DB level with HNSW indexes. O(log n) vs O(n). Scales.
Agent Framework	LangGraph	Raw if/else routing	State machine with conditional edges. Extensible, debuggable, production-correct.
API Framework	FastAPI	FastAPI	Same — it's the right choice for Python APIs
LLM	Gemini	Gemini	Same — good balance of quality/cost
Security	Rate limiting + injection detection + PII masking	None	Production systems need this. Period.
Observability	LangSmith	Disabled	Can't debug what you can't see
Embeddings	Google gemini-embedding-001	sentence-transformers (all-MiniLM-L6-v2)	Higher quality embeddings, simpler setup

Production API Request Flow

```
Client Request
    |
    v
[Rate Limiter] ---------> slowapi, tracks requests per IP address
    |
    v
```

```
[Security Pipeline] ----> InputSanitizer + PII Detector/Masker + Injection Detector
    |
    v
[Cache Layer] ----------> SHA256 hash key + TTL. Hit? Return cached. Miss? Continue.
    |                     (In-memory for demo. Redis in production.)
    v
[LangGraph Agent] ------> The brain. Primary model -> retry -> fallback -> graceful error.
    |                     User NEVER sees a stack trace.
    v
[Output Validator] -----> Validates response structure (Pydantic)
    |
    v
[Cache Store] ----------> Save response for future identical queries
    |
    v
[Metrics + Logging] ----> Structured JSON logger + MetricsCollector
    |                     (ELK/Datadog/CloudWatch for logs. Prometheus for metrics.)
    v
JSON Response
```

What I Already Know (from langc-course)

- LLM setup (Gemini, Groq) with LangChain
- Embeddings (Google gemini-embedding-001), cosine similarity, caching
- Document loaders (Text, PDF, Web)
- Text splitters (Recursive, Semantic chunking)
- Basic RAG pipeline with LCEL chains
- Advanced retrieval: Multi-Query, Contextual Compression, Parent Document Retriever
- Hybrid search (BM25 + Vector with custom RRF)
- Production RAG with PGVector/Supabase
- LangSmith observability basics

What I'm Learning New

1. **FastAPI** — Building production APIs with lifespan, middleware, dependency injection, async handlers
2. **LangGraph** — Stateful agents with built-in safety nets (retry -> fallback -> graceful error)
3. **Security Layer** — InputSanitizer + PII Detector/Masker + Injection Detector -> SecurityPipeline
4. **Caching** — TTL-based with SHA256 keys (in-memory for demo, Redis for production)
5. **Structured Logging** — JSON logger (not print!), needed for ELK/Datadog/CloudWatch
6. **Metrics Collection** — MetricsCollector (dictionary demo) -> Prometheus in production
7. **Docker** — Layer caching (deps before code), non-root user, health checks
8. **Deployment** — render.yaml, containerized deployment, Docker as runtime
9. **Production Scaling** — Redis (not in-memory), Prometheus (not dict counters), load balancer in front

Comparison with Akshat's Doc-Analyzer

Where His is Stronger

- Full Next.js frontend with virtualized tables, document preview, multi-tab chat
- Image analysis (multimodal — sends page images to Gemini Vision)
- Table modification ("change CGPA for 2027 to 10" -> rewrites table with download)
- Deployed and usable (Vercel + cloud MongoDB)

Where Ours is Stronger (Architecture)

- LangGraph agent vs raw if/else (extensible, debuggable)
- PGVector with HNSW indexes vs in-memory cosine loop (scalable)
- Full security layer (rate limiting, injection detection, PII masking)
- LangSmith observability from day one
- Clean separation of concerns (not a 1000-line god-class)
- Caching layer (reduces costs, improves latency)
- Output validation with fallback models

Project Structure (Planned)

```
docmind/
├── backend/
│   ├── main.py                # FastAPI app entry point
│   ├── config.py              # Environment & settings
│   ├── middleware/
│   │   ├── rate_limiter.py    # slowapi rate limiting
│   │   ├── security.py        # Injection detection + PII masking
│   │   └── cache.py           # Semantic caching layer
│   ├── api/
│   │   ├── routes/
│   │   │   ├── documents.py   # Upload, process, list documents
│   │   │   ├── chat.py        # Chat endpoints
│   │   │   └── health.py      # Health check
│   │   └── dependencies.py    # Shared dependencies
│   ├── services/
│   │   ├── document_service.py # PDF processing pipeline
│   │   ├── retrieval_service.py# Vector search + hybrid retrieval
│   │   └── chat_service.py    # Chat session management
│   ├── agent/
│   │   └── graph.py           # LangGraph agent definition
│   ├── models/
│   │   └── schemas.py         # Pydantic models
│   └── db/
│       └── supabase.py        # PGVector connection
├── docs/
│   └── notes/                 # Learning notes (this folder)
├── .env.example
├── requirements.txt
└── pyproject.toml
```

01 - Production API Request Flow

What It Is

A layered architecture where every API request passes through multiple middleware layers before reaching the LLM and returning a response. Each layer has a specific job — security, performance, reliability, or observability.

Think of it like airport security: you don't go straight from the entrance to the plane. You pass through check-in, security screening, boarding pass verification, etc. Each layer catches a different class of problem.

Dependencies & Why Each Exists

Package	Purpose
`fastapi`	Web framework — async, type-safe, auto-generates OpenAPI docs
`uvicorn`	ASGI server — actually runs the FastAPI app (like gunicorn but async)
`slowapi`	Rate limiting — wraps `limits` library for FastAPI integration
`pydantic-settings`	Config management — loads env vars into typed Python classes
`langchain` / `langgraph`	LLM orchestration + agent framework

Why pydantic-settings over python-dotenv? `python-dotenv` just loads `.env` into `os.environ` — no validation, no types. `pydantic-settings` gives you typed config classes with validation, defaults, and IDE autocomplete. If a required env var is missing, it fails at startup (not at 2am when a user hits that code path).

The Layers (In Order)

1. Rate Limiter (slowapi)

What: Limits how many requests a user/IP can make per time window.

Why it matters:

- LLM calls cost money (Gemini charges per token)
- Without limits, one user can burn your entire monthly budget in minutes
- Prevents DDoS attacks and abuse
- Standard practice for any production API

How it works:

- Track requests per IP address
- Return HTTP 429 (Too Many Requests) when limit exceeded
- Common patterns: 10 req/min for free tier, 100 req/min for paid

Interview answer: "Rate limiting is essential for production LLM APIs because each request has real cost. We used slowapi (built on top of Python's limits library) to enforce per-user request quotas. This protects against both accidental abuse and intentional DDoS, while also managing our LLM API budget."

2. Security Middleware (The Security Pipeline)

What: Three classes combined into one pipeline:

1. **InputSanitizer** — cleans and validates raw input
2. **PII Detector & Masker** — finds and redacts sensitive data
3. **Injection Detector** — catches prompt injection attempts

These are combined into a single **SecurityPipeline** class that runs all three in sequence.

Interview answer: "Our security middleware is a pipeline of three stages. InputSanitizer handles basic validation and cleaning. The PII Detector uses regex patterns to find and mask sensitive data like credit cards and SSNs before they reach the LLM. The Injection Detector uses pattern matching to catch prompt injection attempts. These are composed into a SecurityPipeline class — single responsibility per class, composed together."

3. Cache Layer (TTL + SHA256)

What: If someone asked the same question before, return the cached answer without calling the LLM.

How it works:

- **SHA256 hash** of the query as the cache key (exact match caching)
- **TTL (Time To Live)** — cached responses expire after a set time
- In-memory dictionary for the course demo
- **In real production:** Redis instead of in-memory

Two types of caching:

1. *Exact match (SHA256)* — hash the query string. Fast, simple, but "What is LangChain?" and "Tell me about LangChain" are different cache keys.
2. *Semantic cache* — embed the query, find similar past queries by cosine similarity. Higher hit rate, more complex.

Interview answer: "We implement response caching with SHA256-hashed query keys and TTL-based expiration. In our demo this is in-memory, but in production you'd use Redis for persistence across instances and horizontal scaling. For higher cache hit rates, you can upgrade to semantic caching where you embed queries and match by similarity."

4. Output Validator

What: The actual LLM call, wrapped with retry logic and fallback models.

How it works:

```
Try primary model -> Success? Validate output structure -> Return
                  -> Failure? Retry
                      -> Still failing? Switch to fallback model
                          -> All failed? Return graceful error message
```

Interview answer: "Our output validator wraps the LLM call with retry logic and model fallback. The primary model is configured for speed and cost. If it fails or returns malformed output, we retry, then fall back to a secondary model. The user never sees a raw error."

5. Metrics + Logging (Structured JSON)

Two components:

1. **Structured JSON Logger** — replaces `print()` statements with proper logging
2. **MetricsCollector** — tracks counters, latencies, rates

Interview answer: "We never use print statements in production. Our structured JSON logger outputs machine-parseable log entries with context (request ID, user, latency, model used). These feed into log aggregators like ELK Stack or CloudWatch for searching and alerting. For metrics, we use a collector that tracks request counts, latencies, and error rates — in production this would be Prometheus with Grafana dashboards."

The LangGraph Agent (The Brain)

How the safety net works:

```
User Query -> Agent Node (primary model)
                |
         Success? -> Return response
                |
         Failure? -> Retry Node
                      |
               Success? -> Return response
                      |
               Failure? -> Fallback Node (secondary model)
                             |
                      Success? -> Return response
                             |
                      Failure? -> Error Message Node
                                    -> "Sorry, service unavailable"
```

main.py — Joining Everything Together

Key pattern: Lifespan function

```python
@asynccontextmanager
async def lifespan(app: FastAPI):
    # STARTUP: create all components
    app.state.agent = ProductionAgent(settings)
    app.state.security = SecurityPipeline(settings)
    app.state.cache = Cache(settings)
    app.state.metrics = MetricsCollector()
    yield
    # SHUTDOWN: log final metrics summary
    app.state.metrics.log_summary()
```

The chat endpoint flow:

1. Rate limit check (slowapi decorator)
2. Security pipeline check -> reject if unsafe
3. Cache lookup -> return if hit
4. LangGraph agent invoke -> get response
5. Output validation -> ensure quality
6. Cache store -> save for future
7. Log metrics -> track everything
8. Return JSON response

Production vs Demo (What Changes at Scale)

Concern	Course Demo	Real Production
Caching	In-memory dictionary	**Redis**
Metrics	Dictionary counters	**Prometheus** + Grafana
Rate limiting	In-memory (per instance)	**Redis-backed**
Load balancing	Single instance	**Nginx/ALB** in front
Secrets	`.env` file	**Vault/AWS Secrets Manager**
Logging	stdout	**ELK Stack / Datadog / CloudWatch**

Key Interview Questions

Q: Why not just call the LLM directly? A: In production, you need reliability, security, cost control, and observability. The middleware layers provide: abuse prevention (rate limiting), data protection (security), cost reduction (caching), reliability (output validation + fallback), and debuggability (metrics/logging).

Q: What happens when your primary LLM is down? A: LangGraph agent safety net: Primary model fails -> retry -> fallback model -> graceful error message. Combined with caching, many requests can still be served during an outage.

Q: How do you reduce LLM costs in production? A: (1) Caching with TTL. (2) Rate limiting. (3) Model routing — cheaper models for simple queries. Together these reduce costs 50-70%.

Q: How would you horizontally scale this? A: Move all shared state to external services: Redis for caching and rate limiting, Prometheus for metrics, PostgreSQL for persistent data. Run N instances behind a load balancer. Application code doesn't change.

02 - FastAPI Fundamentals

What It Is

FastAPI is a modern Python web framework for building APIs. It's async-first, uses Python type hints for automatic validation, and auto-generates interactive API docs (Swagger UI at `/docs`).

Why FastAPI (Not Flask, Not Django)

	FastAPI	Flask	Django
Async	Native (async/await)	Needs workarounds	Limited
Performance	Very fast (built on Starlette)	Slower	Slower
Type safety	Built-in (Pydantic)	Manual	Partial
API docs	Auto-generated	Manual (Swagger plugin)	Manual
Learning curve	Low	Low	High
Use case	APIs, microservices	Small apps, APIs	Full web apps with ORM

Interview answer: "FastAPI is ideal for LLM/AI backends because it's async-native (LLM calls are I/O-bound, async lets us handle many concurrent requests), has built-in Pydantic validation (we validate every request and response automatically), and auto-generates OpenAPI docs so frontend teams always have an up-to-date API reference."

Key Concepts

Lifespan (Modern Startup/Shutdown)

```
from contextlib import asynccontextmanager

@asynccontextmanager
async def lifespan(app: FastAPI):
    # STARTUP
    app.state.agent = ProductionAgent(settings)
    app.state.cache = Cache(settings)
    yield  # App is running
    # SHUTDOWN
    app.state.metrics.log_summary()

app = FastAPI(lifespan=lifespan)
```

app.state — Sharing Components Across Requests

```
@app.post("/chat")
async def chat(request: Request):
```

```
    agent = request.app.state.agent  # Created once at startup
    cache = request.app.state.cache  # Not re-created per request
```

Pydantic Models for Request/Response

```
from pydantic import BaseModel

class ChatRequest(BaseModel):
    message: str
    session_id: str | None = None

class ChatResponse(BaseModel):
    response: str
    cached: bool
    model_used: str
    latency_ms: float
```

pydantic-settings for Configuration

```
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    primary_model: str = "gemini-2.0-flash"
    fallback_model: str = "gemini-1.5-pro"
    google_api_key: str  # Required — app won't start without it
    rate_limit: str = "10/minute"
    cache_ttl: int = 3600

    class Config:
        env_file = ".env"
```

Interview Questions

Q: Why async for an LLM API? A: LLM calls are I/O-bound (1-5 seconds waiting). With async, while waiting for the LLM, the server handles other requests. Dramatically improves throughput under concurrent load.

Q: What's the difference between Uvicorn and FastAPI? A: FastAPI is the web framework (routing, validation). Uvicorn is the ASGI server that handles HTTP connections. Similar to Flask needing Gunicorn.

Q: How do you share expensive resources across requests? A: FastAPI's lifespan pattern. Create objects once at startup, store on `app.state`. Every request handler accesses them — no per-request initialization.

03 - LangGraph Agent (The Brain with a Safety Net)

What It Is

LangGraph is a library for building stateful, multi-step LLM applications as **graphs**. Each step is a **node**, and the flow between steps is controlled by **edges** (which can be conditional).

Why LangGraph (Not Just a Raw LLM Call)

The key insight: Error handling is part of the architecture, not an afterthought wrapped in try/except.

Core Concepts

State

```
class AgentState(TypedDict):
    query: str
    response: Optional[str]
    model_used: Optional[str]
    retries: int
    error: Optional[str]
```

Nodes (Functions that process state)

```
def call_primary_model(state: AgentState) -> AgentState:
    try:
        response = primary_llm.invoke(state["query"])
        return {**state, "response": response, "model_used": "primary"}
    except Exception as e:
        return {**state, "error": str(e), "retries": state["retries"] + 1}
```

Conditional Edges (Routing logic)

```
def should_retry_or_fallback(state: AgentState) -> str:
    if state.get("response"):
        return "done"
    elif state["retries"] < 2:
        return "retry"
    else:
        return "fallback"
```

Building the Graph

```
from langgraph.graph import StateGraph, END

graph = StateGraph(AgentState)
```

```
graph.add_node("primary", call_primary_model)
graph.add_node("fallback", call_fallback_model)
graph.add_node("error", error_response)

graph.set_entry_point("primary")
graph.add_conditional_edges("primary", should_retry_or_fallback, {
    "done": END,
    "retry": "primary",
    "fallback": "fallback"
})
graph.add_conditional_edges("fallback", should_return_or_error, {
    "done": END,
    "error": "error"
})
graph.add_edge("error", END)

agent = graph.compile()
```

LangGraph vs Akshat's Approach

Akshat: Raw if/else routing. Adding a new chat type means modifying existing code. Error handling is scattered try/except blocks.

LangGraph: Adding a new chat type = adding a node + edge. Error handling is structural. Visually debuggable via LangSmith.

Graphs vs Chains

	LCEL Chains	LangGraph
Flow	Linear (A -> B -> C)	Graph (branches, loops, conditions)
Error handling	Try/except wrappers	Built into graph structure
Cycles	Not possible	Supported (retry loops)
Use case	Simple RAG pipeline	Agents, multi-step workflows

Interview Questions

Q: What is LangGraph and why did you use it? A: LangGraph builds stateful LLM apps as graphs. We needed conditional routing (retry vs fallback), cycles (retry loops), and explicit state management — impossible with linear LCEL chains.

Q: How does your agent handle LLM failures? A: Built-in safety net: primary node -> conditional edge -> retry (up to N) -> fallback model -> error response. The user never sees a stack trace.

Q: How do you add a new capability? A: Add a node + edge. Existing nodes untouched. Open/closed principle.

04 - Docker & Deployment

Dockerfile — Key Decisions

```
FROM python:3.12-slim
WORKDIR /app

# Dependencies FIRST (Docker layer caching!)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Application code SECOND
COPY . .

# Non-root user (security)
RUN useradd -m appuser
USER appuser

# Health check
HEALTHCHECK --interval=30s --timeout=10s --retries=3 \
    CMD curl -f http://localhost:8000/health || exit 1

EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Why This Order Matters (Layer Caching)

```
First build:  ~4 minutes (downloads everything)
Second build (only code changed):
  Layer 1-3: CACHED (requirements didn't change)
  Layer 4: REBUILDS (code changed)
  Total: ~10 seconds
```

Three Key Decisions

1. **Dependencies before code** — Docker layer caching saves minutes on rebuilds
2. **Non-root user** — Limits blast radius if container is compromised
3. **Health check** — Orchestrators auto-restart unhealthy containers

Deployment (Render)

```
services:
  - type: web
    name: docmind-api
    runtime: docker
    envVars:
```

```
      - key: GOOGLE_API_KEY
        sync: false
```

Push to GitHub -> Render builds Docker image -> Deploys with HTTPS.

Interview Questions

Q: Explain Docker layer caching. A: Docker caches each layer. We install deps BEFORE copying code. Since deps change rarely, most builds skip the expensive pip install. Rebuilds: 5 min -> 10 sec.

Q: Why non-root user? A: Principle of least privilege. Limits attacker access if container is compromised.

Q: How would you scale this API? A: Horizontally — multiple containers behind a load balancer. Move cache/metrics/rate-limiting to Redis/Prometheus (external services shared across instances).

05 - Security Layer

The Three Classes

1. **InputSanitizer** — cleans/validates raw input (length, encoding, control chars)
2. **PII Detector & Masker** — regex patterns for credit cards, SSNs, emails, phones
3. **Injection Detector** — pattern matching against known injection phrases

Combined into a **SecurityPipeline** class.

PII Masking Example

```
Input:  "My credit card is 4532-1234-5678-9012 and email is john@gmail.com"
Output: "My credit card is [REDACTED_CC] and email is [REDACTED_EMAIL]"
```

Injection Detection

Common patterns: "ignore previous instructions", "repeat your system prompt", "you are now...", "jailbreak"

Detection uses scoring — single match might be innocent, multiple = likely attack.

SecurityPipeline Flow

```python
class SecurityPipeline:
    def process(self, raw_input: str) -> SecurityResult:
        sanitized = self.sanitizer.clean(raw_input)
        injection_result = self.injection_detector.detect(sanitized)
        if injection_result.is_blocked:
            return SecurityResult(blocked=True, reason="Injection detected")
        masked_input = self.pii_masker.mask(sanitized)
        return SecurityResult(blocked=False, cleaned_input=masked_input)
```

Interview Questions

Q: How do you protect against prompt injection? A: Defense in depth: (1) Input-side pattern matching + scoring. (2) Strong system prompts with boundaries. (3) Output validation. No single layer is perfect — stack them.

Q: Can you guarantee injection prevention? A: No. There's no clear boundary between "instruction" and "data" for LLMs. You reduce attack surface: detect patterns, limit model capabilities, validate outputs, monitor anomalies.

Q: Why three classes instead of one? A: Single Responsibility. Each is independently testable and reusable. Composed in the pipeline.

06 - Caching & Monitoring

Caching Implementation

```python
class ResponseCache:
    def __init__(self, ttl: int = 3600):
        self.cache = {}
        self.ttl = ttl

    def _make_key(self, query: str) -> str:
        return hashlib.sha256(query.encode()).hexdigest()

    def get(self, query: str) -> str | None:
        key = self._make_key(query)
        if key in self.cache:
            response, timestamp = self.cache[key]
            if time.time() - timestamp < self.ttl:
                return response
        return None

    def set(self, query: str, response: str):
        key = self._make_key(query)
        self.cache[key] = (response, time.time())
```

Structured JSON Logging

Never print() in production. Log aggregators need JSON:

```json
{
    "level": "ERROR",
    "message": "LLM call failed",
    "request_id": "abc-123",
    "model": "gemini-2.0-flash",
    "latency_ms": 5200,
    "error_type": "timeout"
}
```

MetricsCollector — What to Track

Metric	Why
Request count	Traffic volume
Error rate	System health
Latency (p50, p95, p99)	User experience
Cache hit rate	Cost efficiency

Token usage	Cost tracking
Model usage breakdown	Fallback frequency

Production Tools

Demo	Production
In-memory cache	**Redis**
Dict counters	**Prometheus** + Grafana
print()	**ELK Stack / Datadog / CloudWatch**

Interview Questions

Q: Why structured JSON logging? A: Log aggregators can't parse print(). JSON lets you search "all errors for user X with latency > 3s in the last hour." Finding bugs: minutes vs days.

Q: Caching tradeoffs? A: Main issue is staleness. Manage with TTL and cache invalidation on data changes. Exact-match misses paraphrases.

Q: How to set up alerting? A: Prometheus rules or Datadog monitors. Critical: error rate > 5%, p99 > 10s, fallback rate > 20%. Warning: cache hit < 30%, token spikes.

07 - Streaming, Multi-Doc Chat, Contextual Compression, Evaluation

Streaming Responses (Server-Sent Events)

What: Tokens appear as generated instead of waiting 3-5s for full response.

Protocol: SSE — `Content-Type: text/event-stream`, each chunk is `data: {"token": "...", "done": false}\n\n`

Implementation: FastAPI `StreamingResponse` + LangChain's `llm.astream(messages)`. Final event has `done: true` + metadata.

Why SSE over WebSockets: Simpler, works through load balancers/CDNs, HTTP-based. Good enough for unidirectional streaming.

Interview: "We use SSE with FastAPI StreamingResponse and LangChain's async streaming. First token appears within 200ms. The full pipeline (security, retrieval) runs before streaming starts. Each token is a JSON event; final event includes model used, latency, and sources."

Multi-Document Chat

What: Query across multiple documents in one request. "Compare doc A vs doc B."

How: `retrieve_multi()` runs hybrid retrieval per document, pools all results, re-ranks with RRF. Chunks carry source metadata for citation.

Interview: "We run hybrid retrieval independently on each document, pool all results, then apply RRF. The most relevant chunks from ANY document float to the top. Source metadata lets the LLM cite which document each fact comes from."

Contextual Compression

What: After retrieval, extract ONLY the query-relevant sentences from each chunk. Throw away noise.

The problem: Chunks are 1000 chars, maybe 2 sentences matter. Rest wastes tokens and confuses the LLM.

How: One LLM call per chunk: "Extract ONLY sentences relevant to this question." If nothing relevant → chunk removed. Falls back to original on failure.

Trade-off: Extra ~500ms per chunk for significantly cleaner context.

Interview: "After hybrid retrieval, we compress chunks by extracting only query-relevant sentences via a lightweight LLM call. Typically compresses to 30-40% of original — less noise, fewer tokens, better answers. Graceful degradation: if compression fails, we use the original chunk."

Evaluation Pipeline

What: Automated RAG quality scoring. Answers: "Is my RAG actually working well?"

Two scores:

- Retrieval Relevance (1-5): Did we find the right chunks?

- Answer Faithfulness (1-5): Is the answer grounded (not hallucinated)?

How: Generate synthetic Q&A from doc → run through pipeline → score with LLM-as-judge.

When to use: After upload (verify), after parameter changes (compare), as regression test.

Interview: "We have automated evaluation measuring retrieval relevance and answer faithfulness (both 1-5). Synthetic Q&A generation + LLM-as-judge scoring. Most RAG systems have zero quality measurement — ours quantifies the impact of any change and catches regressions."

Key Questions

Q: Faithfulness vs Relevance? A: Relevance = retrieval stage (right chunks?). Faithfulness = generation stage (answer grounded in context?). Need both high. Can have great chunks but hallucinated answer (low faithfulness), or irrelevant chunks but good general answer (low relevance).

Q: When to use compression? A: When chunks are large, context window is limited, or precision matters more than latency. Skip for simple/fast queries where speed is priority.

Self-Correcting RAG (Agentic Retrieval)

What: Retrieval grades its own quality. If chunks aren't relevant, reformulates the query and retries.

Flow:

```
Retrieve → Grade ("do these chunks answer the question?") → YES → generate answer
                                                          → NO → reformulate query → retrieve
again (max 2)
```

Implementation: Three methods — `retrieve_with_correction()`, `_grade_retrieval()` (YES/NO judge), `_reformulate_query()` (rephrase with synonyms/expansion).

Cost: +0.5-1s per query (grading). Only poor retrievals (~20%) pay the reformulation cost (+1-2s more).

Interview: "After hybrid search, an LLM grades the chunks: 'Does this answer the question?' If NO, it reformulates using different keywords and retries — max 2 attempts. Catches vocabulary mismatch that regular RAG passes through silently. 80% of queries pass first time. The 20% that would have failed get self-corrected."

Q: Why not just retrieve more chunks? A: Higher K adds noise. Self-correction is targeted — reformulates for what's actually needed. Precision over recall.

Multi-Provider LLM Architecture

What: Different providers for different tasks. Groq for generation (fast, 30 req/min). Google for embeddings (no Groq alternative).

Implementation: `_create_llm()` factory. Swap provider in config, all components switch.

Interview: "We use Groq for generation (10x faster than Google, generous limits) and Google for embeddings only. A factory function abstracts the provider — one config change to switch. Production systems mix providers to optimize cost/speed/quality per task."

Q: What if Groq goes down? A: LangGraph safety net: primary (Llama 70B) → retry → fallback (Llama 8B) → graceful error. Could add Google as cross-provider fallback.